

Copilot Instructions

Project Overview

Amzur AI Chat is an internal multi-user conversational AI platform. It provides threaded persistent chat, email/password and Google OAuth authentication, conversational memory, multi-modal input (images, video, code, PDF), AI image generation, RAG over uploaded documents, and natural language querying of databases and spreadsheets.

All AI calls route exclusively through the Amzur LiteLLM proxy at litellm.amzur.com. Direct calls to any AI provider (OpenAI, Google, Anthropic) are not permitted.

Tech Stack

Layer	Technology
Frontend	React 18+, TypeScript, Tailwind CSS
Backend	FastAPI, Python 3.11+
Database	PostgreSQL — SQLAlchemy 2.0, Alembic migrations
AI Orchestration	LangChain (LCEL)
LLM Gateway	Amzur LiteLLM Proxy (litellm.amzur.com) — all model and embedding calls
Models	gpt-4o , gemini/gemini-2.5-flash (via proxy)
Embeddings	text-embedding-3-large (via proxy)
Vector Store	ChromaDB (persisted to disk)
Auth	Email/password (bcrypt + JWT) + Google OAuth 2.0
File Handling	Images, video, PDF, Excel, Google Sheets

Repository Structure

```
None
/
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   │   ├── chat/           # MessageList, InputBar, ThreadSidebar
│   │   │   ├── attachments/   # File/image/video upload components
│   │   │   └── auth/          # Login, OAuth callback
│   │   ├── pages/
│   │   ├── hooks/
│   │   ├── lib/               # API client, auth helpers, utilities
│   │   └── types/             # Shared TypeScript interfaces
│   └── backend/
│       ├── app/
│       │   ├── api/           # Routers - HTTP only, no business logic
│       │   ├── services/      # All business logic
│       │   ├── models/        # SQLAlchemy ORM models
│       │   ├── schemas/       # Pydantic request/response schemas
│       │   ├── ai/
│       │   │   ├── llm.py      # LiteLLM client singletons - import from here
│       │   │   ├── chains/     # LCEL chains, one file per feature
│       │   │   ├── memory/     # Conversation memory utilities
│       │   │   ├── rag/        # ChromaDB client, ingestion, retrieval
│       │   │   └── prompts/    # All prompt templates (.txt / .yaml)
│       │   ├── db/            # Session factory, Alembic env
│       │   └── core/           # Settings, logging, config
```

Frontend Conventions

Components

- Functional components and hooks only — no class components
- Named exports for all components; default exports for page-level components only
- PascalCase filenames for components (`ChatMessage.tsx`); camelCase for hooks (`useThreadList.ts`)
- Single responsibility — split any component exceeding ~150 lines

Tailwind CSS

- Utility classes inline in JSX — no custom CSS files unless Tailwind cannot handle it
- No `@apply` outside shared design-system components
- Standard spacing scale — no arbitrary values (`mt-[13px]`, `w-[347px]`, etc.)
- Dark mode via `dark:` variant — no JS-managed theme toggling

Rendering Conventions

- Message content rendered via `react-markdown` — never render raw HTML strings from the API
- Streaming responses render token-by-token — never buffer until completion
- Support markdown, syntax-highlighted code blocks, and LaTeX in message output
- Chat message bubbles must use `text-justify` (Tailwind) — never center-aligned text

State Management

- Server state: TanStack Query (React Query) — no `fetch` calls in `useEffect`
- Local UI state: `useState` / `useReducer`
- Auth/global state: Zustand or React Context

API & Types

- All API calls through `/src/lib/api.ts` — never call `fetch` or `axios` directly in components
- All API response shapes defined in `/src/types/` and imported from there
- TypeScript strict mode — `any` is a type error
- Runtime validation of API responses with `zod` where response shape is uncertain

Backend Conventions

Layered Architecture

Every feature follows this separation — no exceptions:

- **Routers** (`/api/`): parse request → call service → return response. No logic, no DB access.
- **Services** (`/services/`): all business logic. Framework-agnostic. Fully unit-testable.
- **Models** (`/models/`): SQLAlchemy ORM definitions only. No methods, no logic.
- **Schemas** (`/schemas/`): Pydantic I/O models. Always separate from ORM models.

When in doubt about where logic belongs, it belongs in the **service**.

FastAPI

- `async def` for all route handlers
- `Depends()` for DB sessions, auth, and shared services — never instantiate inside handlers
- All routes declare explicit `response_model`
- Streaming via `StreamingResponse(media_type="text/event-stream")`
- Structured error responses:

Python

```
raise HTTPException(
    status_code=404,
    detail={"error": "not_found", "message": "Resource not found"}
)
```

PostgreSQL + SQLAlchemy

- SQLAlchemy 2.0 style (`select()`, mapped columns) — no legacy 1.x patterns
- Schema changes via Alembic migrations only — never modify the DB directly
- UUID primary keys on all tables
- `DateTime(timezone=True)` on all timestamps — store UTC, convert at the API boundary
- No N+1 queries — use `selectinload` or `joinedload` for related data
- Feature-specific optional settings typed as `Optional[str] = None` in `config.py` so the app boots without all env vars present

Auth

Two strategies share a single JWT layer. Token structure, `get_current_user`, and the `httpOnly` cookie are identical for both — adding a new strategy does not change anything downstream.

Email / Password

- Hash passwords with bcrypt on write — never store plaintext
- Verify hash on login, issue JWT as `httpOnly` cookie (`samesite="lax"`, `secure=False` in dev, `secure=True` behind HTTPS)

Google OAuth 2.0

- Redirect to Google → exchange code → extract profile → issue same JWT cookie → redirect to frontend
- **Account linking:** if Google email matches an existing user, populate `google_id` — never create a duplicate user
- Google-only accounts carry a null `hashed_password`

JWT

- Signed with `SECRET_KEY` from environment variables, expiry from `JWT_EXPIRE_MINUTES`
 - Stored exclusively in `httpOnly` cookie — never in `localStorage`, response body, or React state
 - `get_current_user` reads from cookie via `Depends()` — no inline auth checks in route handlers
-

AI Layer

LiteLLM — Single Gateway (`/ai/llm.py`)

`litellm.amzur.com` is the only permitted entry point for all AI calls. Import clients from `/ai/llm.py` — never instantiate them elsewhere.

LangChain LLM (for chains):

```
Python
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(
    model=settings.LLM_MODEL,
    base_url=settings.LITELLM_PROXY_URL,
    api_key=settings.LITELLM_API_KEY,
    timeout=30,
    max_retries=2,
)
```

OpenAI SDK client (for direct calls — image gen, embeddings):

Python

```
from openai import OpenAI

client = OpenAI(
    api_key=settings.LITELLM_API_KEY,
    base_url=settings.LITELLM_PROXY_URL,
)
```

Embeddings:

Python

```
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings(
    model=settings.LITELLM_EMBEDDING_MODEL,
    base_url=settings.LITELLM_PROXY_URL,
    api_key=settings.LITELLM_API_KEY,
)
```

Usage tracking — required on every call: Every AI call must include the authenticated user's email for cost attribution and budget enforcement.

Python

```
# Direct SDK call
client.chat.completions.create(
    model=settings.LLM_MODEL,
    messages=[...],
    user=current_user.email,
    extra_body={
        "metadata": {
            "application": settings.APP_NAME,
            "environment": settings.ENVIRONMENT,
        }
    }
)

# LangChain chain invocation
chain.invoke(
    {"human_input": message, "history": history},
    config={"metadata": {"user_email": current_user.email}}
)
```

Error handling:

```
Python
from openai import OpenAIError

try:
    response = client.chat.completions.create(...)
except OpenAIError as e:
    raise HTTPException(status_code=502, detail={"error": "llm_error", "message":
str(e)})
except Exception as e:
    raise HTTPException(status_code=500, detail={"error": "unexpected", "message":
str(e)})
```

Available models:

- Chat: `gpt-4o`, `gemini/gemini-2.5-flash`
- Embeddings: `text-embedding-3-large`
- Image generation: `gemini/imagen-4.0-fast-generate-001` **Usage dashboard:**
<https://litellm.amzur.com/ui>

LangChain

- LCEL syntax: `prompt | llm | parser` — no `LLMChain`, `SequentialChain`, or `ConversationalRetrievalChain`
- Prompt templates in `/ai/prompts/` as `.txt` or `.yaml` — never inline strings in chain definitions
- All user-facing LLM responses streamed — never block on full completion
- Multi-step or stateful flows use `LangGraph` — not sequential chains
- All AI logic in `/ai/` and `/services/` — never in route handlers

File & Attachment Handling

- Files saved to `UPLOAD_DIR` on disk — never stored as DB blobs
- DB records path, MIME type, original filename, and type classification only
- MIME type validated server-side on upload — file extension is not trusted
- File size enforced via `MAX_UPLOAD_MB` on both frontend and backend

- Accepted MIME types defined in `settings` — never hardcoded in route handlers or service functions
-

Environment Variables

None

App

SECRET_KEY=

JWT_EXPIRE_MINUTES=480

APP_NAME=amzur-ai-chat

ENVIRONMENT=development

Database

DATABASE_URL=postgresql+asyncpg://user:pass@host:5432/dbname

Amzur LiteLLM Proxy

LITELLM_PROXY_URL=https://litellm.amzur.com

LITELLM_API_KEY=sk-

LLM_MODEL=gemini/gemini-2.5-flash

LITELLM_EMBEDDING_MODEL=text-embedding-3-large

IMAGE_GEN_MODEL=gemini/imgen-4.0-fast-generate-001

Google OAuth

GOOGLE_CLIENT_ID=

GOOGLE_CLIENT_SECRET=

GOOGLE_REDIRECT_URI=http://localhost:8000/api/auth/google/callback

ChromaDB

CHROMA_PERSIST_DIR=./chroma_db

Google Sheets

GOOGLE_SERVICE_ACCOUNT_JSON=

File uploads

MAX_UPLOAD_MB=20

UPLOAD_DIR=./uploads

Testing

- **Backend:** `pytest` + `pytest-asyncio`; `httpx.AsyncClient` for route integration tests; isolated test DB
 - **Frontend:** Vitest + React Testing Library
 - **Naming:** `test_<module>.py` / `<Component>.test.tsx`
 - Every service function has a unit test
 - AI chains: mock LiteLLM responses — no real API calls in CI
 - RAG: fixed ChromaDB fixture — no re-embedding per test run
-

Security

- No secrets, API keys, or model names hardcoded — environment variables only
 - All user input validated by Pydantic schemas before reaching services
 - NL-to-SQL: read-only enforced, case-insensitive keyword block, restricted table scope
 - Auth via `Depends(get_current_user)` only — no inline auth checks
 - MIME type validated server-side on all file uploads — extension not trusted
 - No raw LLM prompt content or AI responses containing PII written to logs
-

Git & Code Quality

- Conventional Commits: `feat:`, `fix:`, `chore:`, `docs:`, `refactor:`
 - Pre-commit hooks: `ruff` (Python linting + formatting), `eslint` + `prettier` (TypeScript/JSEX)
 - All Python functions carry type annotations
 - PRs must not introduce linting errors or failing tests
-

Copilot Directives

These are standing instructions. Apply them on every generation, regardless of what the prompt asks for.

- Router → service → schema → model. If logic appears in a router, move it to the service.
- LCEL only for LangChain chains. Never generate `LLMChain`, `SequentialChain`, or `ConversationalRetrievalChain`.

- Every AI API call includes `user=current_user.email` or `config={"metadata":{"user_email": ...}}`. No exceptions.
 - All AI calls use `settings.LITELLM_PROXY_URL` and `settings.LITELLM_API_KEY`. Never generate a direct OpenAI, Google, or Anthropic API call.
 - `OpenAIEmbeddings` always constructed with `base_url=settings.LITELLM_PROXY_URL` — never the default endpoint.
 - JWT in `httpOnly` cookie only. Never `localStorage`, never the response body.
 - Files to disk. Paths to DB. Never store file content as a database blob.
 - NL-to-SQL keyword block must be case-insensitive. Always include all six: INSERT, UPDATE, DELETE, DROP, TRUNCATE, ALTER.
 - `SQLDatabase` requires `postgresql+psycopg2://` — always use `_build_sync_db_url()` to convert from `asyncpg`.
 - `return_intermediate_steps=True` on all agents (SQL and Pandas) — required to extract generated query from output.
-

Architecture Decisions

Significant decisions that affect the whole codebase. Understand these before changing anything they govern.

AD-01 — Single AI gateway All AI calls route through `litellm.amzur.com`. This centralises cost tracking, rate limiting, and provider switching. Adding a new model or switching providers requires only an env var change — no code changes.

AD-02 — JWT in httpOnly cookie, not Authorization header The frontend is a browser app. `httpOnly` cookies are inaccessible to JavaScript, which eliminates the most common token exfiltration vector (XSS). Stateless API consumers using Authorization headers are not a current requirement.

AD-03 — Two-strategy auth, one JWT layer Email/password and Google OAuth share identical token structure, cookie setup, and `get_current_user`. Adding a third strategy (e.g. Microsoft OAuth) requires only a new service function and two new routes — nothing downstream changes.

AD-04 — Per-user ChromaDB collections Each user's embedded documents are stored in an isolated collection (`user_{user_id}`). This prevents cross-user document retrieval and allows per-user collection management (deletion, re-indexing) without affecting other users.

AD-05 — Memory from DB, not in-process Conversational memory is fetched fresh from the database on every request. In-process memory stores don't survive server restarts and create

correctness issues under concurrent load. The performance cost (one extra DB query per chat message) is acceptable.

AD-06 — Synchronous driver for LangChain SQL agent LangChain's `SQLDatabase` uses SQLAlchemy's synchronous reflection path. A separate `psycpg2` driver and URL conversion is required. The FastAPI async path continues to use `asyncpg` unchanged.

Known Issues

Confirmed bugs and environment-specific issues. Check this section before debugging anything in the affected areas.

KI-01 — Vite scaffolding via npm `npm create vite@latest` swallows the `--template` flag and triggers an interactive prompt. → Use `npx create-vite@latest frontend --template react-ts` instead.

KI-02 — New thread UX flicker Changing `activeThreadId` in `useChat.ts` triggers a `useEffect` that clears messages, wiping an in-progress streamed response. `setActiveThreadId` in `ChatPage.tsx` must fire immediately after `createThread` resolves, before `sendMessage` is called. Suppress the message-clearing effect for the first render after thread creation using a `justCreatedRef` flag.

KI-03 — LiteLLM proxy requires VPN `litellm.amzur.com` is an internal endpoint. `httpx.ConnectError: getaddrinfo failed` means the machine is not on the Amzur VPN. → Connect to VPN. Verify with `nslookup litellm.amzur.com`.

KI-04 — psycpg2 not in requirements LangChain's SQL agent imports `psycpg2` at runtime. `ModuleNotFoundError: No module named 'psycpg2'` on agent import means `psycpg2-binary` is missing from `requirements.txt`. → Add `psycpg2-binary` to `requirements.txt`.

KI-05 — gspread separate from google-auth `ModuleNotFoundError: No module named 'gspread'` occurs even when `google-auth` is installed. → `gspread` is a separate package. Add it explicitly to `requirements.txt`.

KI-06 — RAG answers not persisted If `stream_rag_response` does not receive `db`, `user_uuid`, and `thread_id` parameters, RAG answers are visible during streaming but lost on refresh. The RAG service must call `save_message` for both the user question and the assembled assistant response, matching the chat service pattern.

KI-07 — PowerShell multi-line Python commands Newlines in PowerShell `-c "..."` strings trigger multi-line input mode (`>>`), hanging indefinitely. → Use single-line Python one-liners or run each `python -c "..."` invocation separately.